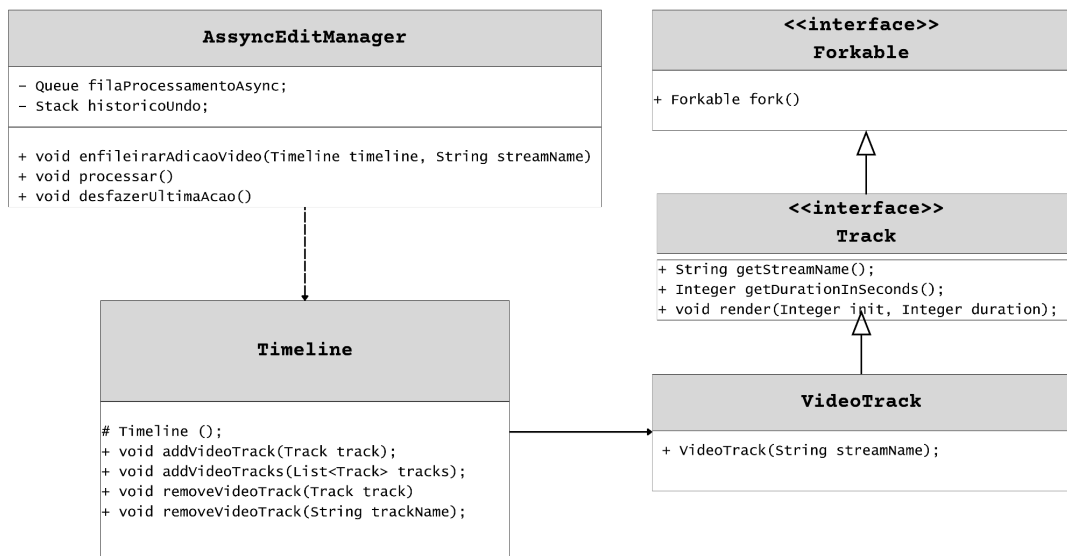


O framework de edição e streaming de vídeo atingiu sua fase final de polimento. Com a consolidação da arquitetura que separou o domínio comercial do técnico, os nossos usuários mais avançados (editores de vídeo e produtores de conteúdo) começaram a estressar o sistema criando projetos de alta complexidade. Durante os testes de carga, a equipe de engenharia detectou dois gargalos críticos envolvendo problemas de espaço, durante operações de renderização e problemas de tempo, durante operações de edição.

Sua tarefa nesta avaliação final é aplicar soluções arquiteturais maduras que resolvam estes problemas de desempenho, sem quebrar as abstrações estabelecidas.

Questão I - Para o módulo de edição do framework, foi desenhada uma interface gráfica robusta, onde o usuário pode realizar diversas modificações estruturais na **Timeline** (ex: "Adicionar Trilha de Vídeo", "Aplicar Filtro de Cor", etc.). Contudo, essas operações de recomposição estrutural da **Timeline** são custosas computacionalmente. Se a interface gráfica invocar esses métodos de alteração diretamente na **Timeline**, o aplicativo irá "congelar" aguardando o processamento. Além disso, uma exigência inegociável dos usuários é a capacidade de "Desfazer" (Undo) e "Refazer" (Redo) de forma sequencial qualquer modificação realizada no projeto. Para resolver essas questões (processamento assíncrono e histórico de reversão), o sistema não pode mais executar as chamadas aos métodos da **Timeline** de forma direta e imediata. A equipe de desenvolvimento pensou em diferir a execução destas operações, que passarão a ser realizadas por uma thread, que executa, quando houver disponibilidade de processadores. Esta abordagem permite manter a responsividade da interface do usuário. Assim, eles projetaram a classe **AssyncEditManager** para enfileiramento das operações, ao mesmo tempo que permite empilhá-las para possibilitar o mecanismo de reversão. Observe o diagrama de classes abaixo e escreva a solução, **mostrando como ela seria usada para permitir a adição e remoção de trilhas de vídeo na timeline** e como seria o código cliente que utilizasse a solução proposta. A solução não deve onerar de forma desproporcional o consumo de memória do editor.



Questão II - Na versão atual do framework, após a finalização da edição de um projeto, a classe **Timeline** exporta o conteúdo consolidado através do seu método **make(String name)**, que

retorna um **HDDBinaryWriter**, usado pelo sistema operacional para gravar os dados finais do filme no disco físico do servidor.

Assim, para a distribuição, é necessário a leitura destas informações através da já conhecida classe **HDDBinaryReader**. Usando o expertise adquirido na etapa anterior, a equipe valeu-se de um adapter de classe, para evitar a duplicação dos dados armazenados em disco, e adaptar a interface do **HDDBinaryReader** para a interface **PlayableContent**, utilizada pelo **StreamDispatcher**.

Contudo, durante o lançamento de um título muito aguardado, a equipe foi surpreendida pela queda do servidor por estouro de memória. A análise da equipe de avaliação de desempenho identificou, que embora o adaptador de classe (**StreamingContent**) evite a duplicação do buffer para cada **PlayableContent**, e, adicionalmente garanta a experiência individual de cada cliente, que assiste de forma independente, trechos distintos do filme, a modelagem não garante que não seja trazida para a memória uma cópia do **rawBuffer** de bytes armazenado para cada cliente, que esteja assistindo o filme.

Observe o código destas classes e aplique uma solução que permita resolver este problema, mostrando como **StreamDispatcher** deve usar essa solução para entregar os streamings lidos dos arquivos do servidor

```
class StreamingContent extends HDDBinaryReader implements PlayableContent {
    private Integer pos;
    private Integer taxa;
    public StreamingContent(String streamName, Integer taxa) {
        super(streamName);
        this.pos = 0;
        this.taxa = taxa;
    }
    public String getStreamName() {
        return this.path;
    }
    public Integer getDurationInSeconds() {
        return super.getDuration();
    }
    public void play() {
        if (this.rawBuffer == null)
            this.loadPhysicalSectors();
        //renderizando de this.pos até
        // Math.min(this.pos + this.taxa, this.getDurationInSeconds())
    }
}
```

```
public interface PlayableContent {
    public Integer getDurationInSeconds();
    public void play();
}
```

```
class StreamDispatcher {
    public void run() {
        StreamingContent streamingContent =
            new StreamingContent("Odisseia.film", 500);
        streamingContent.play();
    }
    public static void main(String[] args) {
        new StreamDispatcher().run();
    }
}
```

ATENÇÃO

- Esta avaliação deverá ser escrita em caneta.
- As soluções devem ser fornecidas em código JAVA. Os métodos descritos nas classes fornecidas já foram implementados. Concentre-se na escrita dos métodos das soluções arquiteturas que você propõe.
- Para cada padrão, identifique claramente os padrões, que estão sendo aplicados em cada questão, bem como quais são os papéis (Participantes), assumidos pelas classes do projeto.